

Dynamic Single-Source Shortest Path: Implementation, Comparison, and Experimental Evaluation of Dijkstra, A*

Juncheng Zhao

Motivation: The Real-World Problem

- Dynamic shortest-path planning: after a path is computed, the environment can change
 - for example, a previously free cell becomes blocked
 - and the system must quickly find a valid new route.
 - In this project, the environment is modeled as grid graphs (and additionally validated on OSM road networks), which captures realistic navigation and routing behavior
- Real-world problem is fast route replanning in changing systems,
 - such as robot navigation in partially unknown environments, game AI with moving/appearing obstacles
 - The theoretical core is dynamic SSSP/search: comparing uninformed search (Dijkstra), heuristic search (A* with Manhattan/Euclidean), and repair-style updates after local graph change

•

Motivation: The Interest and Importance of algorithm

- In many applications, recomputing from scratch after every change is too slow or wasteful, especially when updates are frequent.
- Efficient replanning improves responsiveness, safety, and scalability
- The challenge is the trade-off between optimality guarantees and computational efficiency under changing graphs.
- A* performance depends strongly on heuristic quality (e.g., Manhattan vs Euclidean on 4-connected grids), while dynamic updates require deciding whether local repair can outperform full recomputation.

The Algorithms at a Glance

	Dijkstra	A*
Type	Exact	Exact (with admissible heuristic)
Query (single run)	$O((V+E) \log V)$	Worst-case $O((V+E) \log V)$; practical often smaller
Memory	$O(V)$	$O(V)$
Optimality / Quality	Optimal	Optimal if heuristic admissible/consistent
Key param	—	Heuristic choice (Manhattan / Euclidean)

Algorithm: Dijkstra

- How it works:
 - Dijkstra is a classic single-source shortest path algorithm for graphs with non-negative edge weights.
 - It maintains the current best known distance $dist$ from the source to each node, **repeatedly expands the node with the smallest $dist$ from a min-priority queue, and performs edge relaxation on its neighbors.**
 - Once a node is finalized (closed), its shortest distance is guaranteed.
- Key implementation detail:
 - In my project, Dijkstra is the baseline: it does not use goal-direction information, so it is optimal but usually expands more nodes and takes more time.
 -

- Pseudocode:

```
Initialize  $dist[start] = 0$ , others = inf
Push ( $start, 0$ ) into min-heap
while heap not empty:
     $u$  = node with minimum  $dist$ 
    if  $u$  is goal: stop
    for each neighbor  $v$  of  $u$ :
        if  $dist[u] + w(u,v) < dist[v]$ :
             $dist[v] = dist[u] + w(u,v)$ 
             $parent[v] = u$ 
            push/decrease-key( $v, dist[v]$ )
```

Algorithm: A*

- A* uses the evaluation function: $f(n) = g(n) + h(n)$, where $g(n)$ is the true cost from start to node n , and $h(n)$ is a heuristic estimate from n to the goal. It always expands the node with the smallest f , which gives it directional guidance toward the target.
- If h is admissible (and ideally consistent), A* still returns an optimal shortest path, but typically with fewer expansions than Dijkstra.
- Distance heuristics used in my project:
 - Manhattan distance (for 4-connected grid): usually tighter in this setting, so it tends to reduce node expansions significantly.
 - Euclidean distance (grid + OSM context): admissible, but often looser than Manhattan on 4-connected grids, so guidance is weaker.
 - OSM road-network A*: uses node longitude/latitude to compute an approximate straight-line metric distance (in meters) as the heuristic. (in OSM, it does not use grids, it uses road graph: vertex, edge which is close to the real world)

The comparison of two algorithms

- Dijkstra is an uninformed baseline that expands nodes strictly by current path cost, which makes it robust but often computationally expensive.
- A* adds heuristic guidance ($f=g+h$), so it explores states that are both cheap so far and promising toward the goal. On 4-connected grid maps,
 - Manhattan-based A* consistently expands far fewer nodes than both Euclidean A* and Dijkstra, leading to lower runtime;
 - Euclidean A* is admissible but usually looser in this setting, so its advantage is smaller.
-
- On real OSM road networks, A* also outperforms Dijkstra in runtime and expansions, showing that the same algorithmic trend holds beyond synthetic grids.

Experimental Setup

- Environment:
 - Programming language: Python 3
 - Main libraries: matplotlib, pandas, osmnx, **streamlit (for visualization/dashboard)**, plus Python standard library
 - Interpreter: CPython.
- Dataset: syntetic and real-word data
 - Source: procedurally generated by project code (src/generators.py)
 - Structure: 4-connected grids with random obstacles; start and goal fixed
 - Parameters: grid size, obstacle probability p , random seed
- OSM road-network graphs (real-world validation)
 - Source: OpenStreetMap, downloaded via OSMnx (tools/download_osm.py)
 - Current graphs: New Brunswick and Piscataway road networks
 - Structure: road graph

Experimental Setup

- Experimental Methodology

- Parameter variations (synthetic):
-

Grid sizes	50 × 50 100 × 100 150 × 150 200 × 200
Obstacle probabilities	0.10 0.20 0.30
Runs per configuration:	5

- Measurement technique:

- wall-clock runtime (`time.perf_counter`)
- expanded nodes, distance...

- Statistical treatment:

- means and variances are computed;
- plots are used for comparison

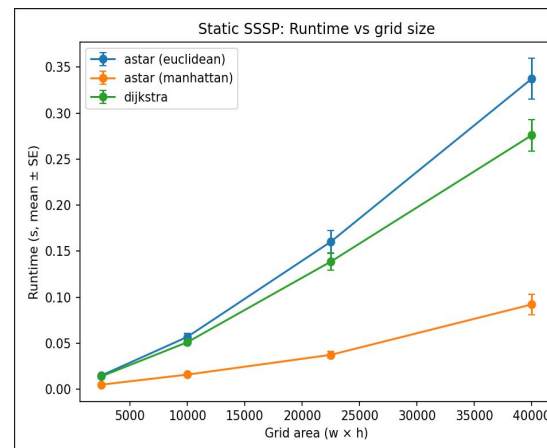
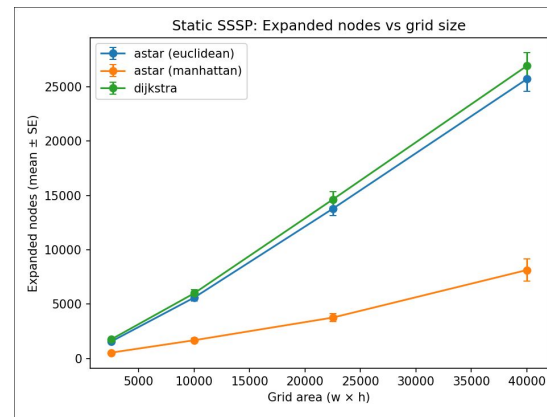
- OSM protocol: evaluate Dijkstra vs A* on multiple sampled start-goal pairs per city graph

- Definitions

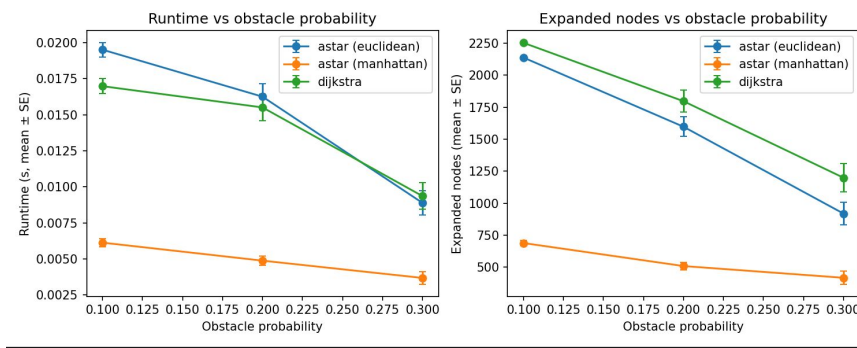
- Runtime: the wall-clock execution time for one shortest-path query (from a start node to a goal node), measured in seconds.
- Expanded nodes: the number of nodes actually popped/processed from the priority queue during search

Results: Synthetic data with grids

- A* (Manhattan) consistently outperforms Dijkstra on 4-connected grids, with lower runtime and significantly fewer expanded nodes.
- A* (Euclidean) often behaves closer to Dijkstra in this setting, and its runtime advantage is limited because the heuristic is looser than Manhattan on 4-connected movement.
- As grid area increases, runtime and node expansions increase for all methods, showing clear scalability pressure from larger state spaces.



Results: Synthetic data outcome with obstacle probability



As obstacle probability increases, average runtime and expansions decrease, since the reachable search space becomes smaller.

Results: OSM(open street map) Result: Real data

Use streamlit to run it in the frontend: core function:

- choose the city(Piscataway vs New Brunswick)
- choose the algorithm
- choose the chart to display

You can now view your Streamlit app in your browser.

Local URL: `http://localhost:8501`

Network URL: `http://10.75.175.109:8501`

OSM Shortest-Path Dashboard

Interactive view for New Brunswick / Piscataway OSM experiments.

Choose city

new_brunswick

Choose algorithm(s)

astar + dijkstra

Quick Conclusion

speedup over Dijkstra

2.34x

Expanded nodes reduction

68.1%

Choose chart(s) to display

☒ Runtime by city

☒ Expanded by city

☒ Runtime vs distance

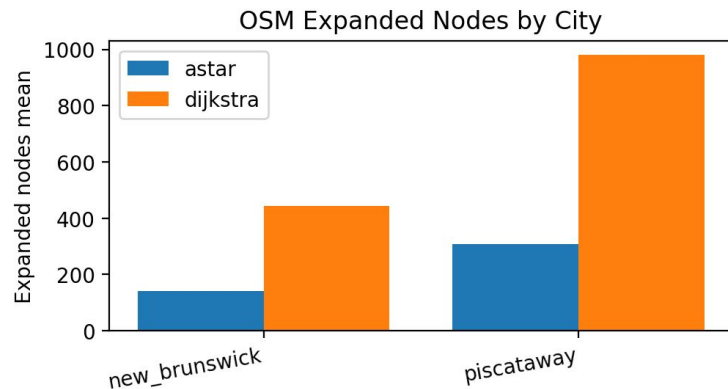
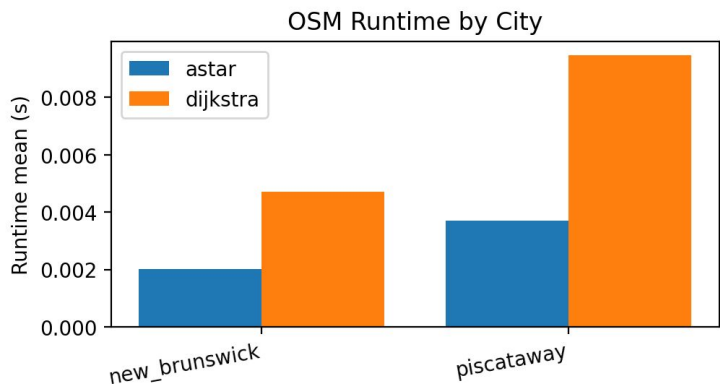
☐ Expanded vs distance

Results: OSM(open street map) Result: Real data

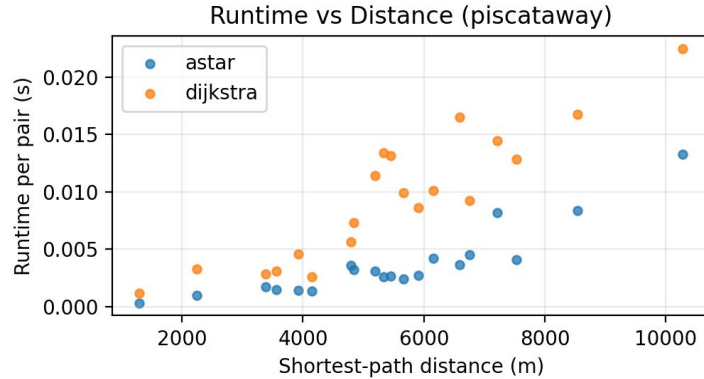
A* is consistently faster than Dijkstra in both OSM cities, as shown by the lower mean runtime bars.

A* expands substantially fewer nodes than Dijkstra in both cities, which explains most of the runtime advantage.

Piscataway is harder than New Brunswick for both algorithms (higher runtime and higher expansions), indicating a larger or more complex effective search space.



Results: OSM(open street map) Result: Real data



In the runtime-vs-distance scatter plot, runtime generally increases with shortest-path distance, and Dijkstra points stay above A* for most OD pairs.

Discrepancies Between Theory and Experiment

- Euclidean performs worse than Manhattan on 4-connected grids: Manhattan is better aligned with grid step movement and provides a tighter heuristic, while Euclidean is looser in this setting, leading to expansion behavior closer to Dijkstra.
- Complexity bounds do not directly capture heuristic quality: Theoretical asymptotic complexity does not explicitly reflect how informative a heuristic is, but empirical performance is highly sensitive to heuristic strength.

Scalability characteristics & Bottlenecks

- scalability characteristics:
 - Synthetic: As grid size increases from 50×50 to 200×200 , the performance gap between algorithms becomes more pronounced, and A (Manhattan)* shows the best scalability.
 - OSM: A* maintains its advantage on real road networks, indicating that the conclusion is not tied to a single data distribution and has external validity.
 - Combined takeaway: Synthetic data provide controlled mechanism-level analysis, while OSM data provide real-world validation.
- bottlenecks:
 - Main bottlenecks: Priority-queue operations and the number of node expansions (e.g., expanded, relaxed).
 - Most “unexpected but reasonable” observation: On grid maps, A (Euclidean) can sometimes be close to, or even slightly worse than, Dijkstra.

Summary of Findings

1. This project confirms that heuristic quality is the key practical driver in shortest-path efficiency: A* with Manhattan consistently outperforms Dijkstra on 4-connected grids in both runtime and node expansions.
2. A* with Euclidean remains correct but is often less competitive on 4-connected grids, showing that an admissible heuristic is not necessarily equally informative.
3. Performance trends are stable across scales: as grid size grows, all methods become slower and expand more nodes, but Manhattan-A* scales best among tested methods.
4. In our setup, increasing obstacle probability reduced average runtime and expansions in our setup, consistent with a smaller reachable search space.
5. Dynamic experiments showed that repair-like updates can help under local changes, but gains are instance-dependent and not universally dominant over rerun.
6. Real-world OSM experiments (New Brunswick, Piscataway) reproduced the same ranking pattern (A* better than Dijkstra), strengthening external validity beyond synthetic grids.

Future Work & Limitations

1. Replace the current repair-like dynamic method with formal incremental planners (e.g., LPA*, D* Lite) for stronger theoretical guarantees and fairer dynamic comparisons.
2. Add explicit memory profiling (peak memory, queue size trajectories) to complement runtime/expansion metrics.
3. Expand dynamic scenarios from single obstacle insertion to multiple sequential updates, including both obstacle addition and removal.
4. Increase OSM diversity (more cities, different road densities) and evaluate robustness under broader topologies and scales.
5. Add weighted traffic-like costs and time-dependent edges to better reflect real routing conditions.

Q&A

Thank you!